

Synopsis Generation for MLOps Project

<https://gemini.google.com/app/e8d68045e67e744d>

User prompt: "I am preparing a formal academic synopsis for my 4-credit postgraduate mini-project. Here is my project metadata: Project Title: Automated MLOps Pipeline for Real-Time Anomaly Detection Domain: MLOps, Linux Systems Engineering, Machine Learning Tech Stack: Python, psutil, scikit-learn (Isolation Forest), FastAPI, Docker, MLflow, Linux systemd. Primary Goal: Monitor Linux system metrics in real time via a systemd daemon, evaluate telemetry using an unsupervised Isolation Forest model, serve predictions through a Dockerized FastAPI endpoint, and track data drift/retraining with MLflow. I will ask you to generate the synopsis section by section following a strict 11-point template. Acknowledge that you understand these parameters and are ready for Topic 1."

Response: I understand the parameters and context for your 4-credit postgraduate mini-project. I am ready to generate your formal academic synopsis section by section following your 11-point template.

Please go ahead with Topic 1.

User prompt: "Generate Section 1: Introduction for the synopsis. Include: Background on modern Linux system monitoring and the limitations of static threshold alerts. The rise of MLOps and unsupervised machine learning in system reliability engineering (SRE). A concise summary of how this proposed automated pipeline bridges telemetry collection and automated model lifecycle management. Keep it academic, formal, and structured in 3 clear paragraphs."

Response: 1. Introduction

Modern Linux-based enterprise environments and distributed cloud infrastructures require robust, continuous system monitoring to maintain high availability and performance. Traditional monitoring systems predominantly rely on rule-based heuristics and static threshold alerting mechanisms (e.g., triggering alerts when CPU utilization exceeds 85% or memory consumption crosses a predefined ceiling). While computationally lightweight, these deterministic approaches suffer from significant limitations in dynamic workloads. They are inherently prone to generating high rates of false positives during routine, transient workload spikes, while failing to detect subtle, multivariate degradation patterns—such as slow memory leaks, zombie process saturation, or abnormal I/O bottlenecks—that stay just below fixed thresholds.

To overcome the fragility of rule-based telemetry monitoring, Modern Site Reliability Engineering (SRE) increasingly incorporates Machine Learning Operations (MLOps) alongside unsupervised learning algorithms. Unsupervised anomaly detection, particularly tree-ensemble methods like Isolation Forests, operates without the need for historically labeled incident datasets. It establishes dynamic operational baselines directly from multidimensional telemetry metrics, isolating irregular system states by measuring path lengths across orthogonal feature partitions. However, applying machine learning to mission-critical infrastructure introduces distinct lifecycle challenges: concept drift caused by evolving workload signatures, pipeline orchestration overhead, and the latency constraints of real-time inference.

This project proposes an end-to-end, automated MLOps pipeline for real-time Linux telemetry anomaly detection that bridges raw system-level data ingestion and continuous model lifecycle governance. Telemetry is collected at the OS layer via a lightweight systemd background daemon using psutil and streamed to a containerized FastAPI microservice hosting an optimized Isolation Forest model. Model versioning, hyperparameter tracking, and automated retraining workflows triggered by data drift are managed centrally via MLflow. By integrating reliable low-level Linux systems programming with automated MLOps pipelines, this framework delivers an adaptive, self-governing reliability solution capable of identifying anomalous system behaviors proactively with minimal manual intervention.

User prompt: "Generate Section 2: Problem Statement. Highlight: The challenge with traditional monitoring tools (high false positive/negative rates, inability to detect multi-variable anomalous interactions). The lack of automated ML retraining pipelines for OS-level telemetry. The risk of delayed incident response during subtle security intrusions or silent memory leaks. Format this with a brief narrative followed by 3–4 bullet points detailing specific pain points."

Response: 2. Problem Statement

Modern Linux system management relies heavily on reactive monitoring tools that fail to capture complex operational failures. As workloads scale dynamically, enterprise systems encounter non-linear metric interactions, micro-burst resource exhaustion, and subtle security threats that bypass standard diagnostic checks. Operating systems lack integrated, real-time machine learning pipelines capable of learning normal baseline behavior, continuously monitoring incoming performance telemetry, and automatically retraining models when underlying workload distributions shift. This operational gap exposes production infrastructure to undetected degraded states and prolonged downtime.

The primary challenges addressed by this project include:

- **Inadequacy of Static Thresholds:** Rule-based alert systems trigger frequent false positives during harmless operational bursts (e.g., scheduled cron jobs) while failing to catch critical anomalies that occur below fixed numerical limits.
- **Inability to Detect Multivariate Metric Interactions:** Complex system degradations—such as silent memory leaks, thread starvation, or subtle privilege escalation attempts—often manifest as simultaneous minor fluctuations across multiple metrics (CPU, I/O, paging) that single-variable monitoring tools treat as benign.
- **Lack of Continuous Model Lifecycle Management:** Deploying static machine learning models to operating system telemetry leads to concept drift and model decay over time, as seasonal usage patterns or software updates alter baseline metrics without triggering continuous retraining mechanisms.

- **Delayed Incident Detection and Response:** Manual analysis of raw Linux system logs and fragmented metric streams increases Time-to-Detect (TTD) and Time-to-Resolve (TTR), allowing performance bottlenecks and zero-day intrusions to persist undetected.

User prompt: Prompt: "Generate Section 3: Objectives. Provide 1 Primary Objective followed by 4–6 Specific/Secondary Objectives (bulleted). Ensure they are measurable and cover data collection (systemd/psutil), model training (Isolation Forest), API deployment (FastAPI + Docker), and experiment tracking (MLflow)."

Response: 3. Objectives

Primary Objective

To design, implement, and evaluate an automated, real-time Linux telemetry monitoring and anomaly detection pipeline that integrates low-level OS metric harvesting with containerized machine learning inference and dynamic lifecycle management using MLOps principles.

Specific / Secondary Objectives

- **Lightweight Telemetry Capture via systemd:** Develop and deploy a persistent Python daemon controlled by a Linux systemd unit file to harvest OS-level telemetry (CPU, memory, disk I/O, and network activity) at sub-second intervals via psutil with under 1% overhead.
- **Unsupervised Anomaly Model Implementation:** Train and validate an unsupervised Isolation Forest algorithm on baseline multi-metric telemetry to isolate multivariate performance anomalies without relying on pre-labeled failure datasets.
- **Microservice Containerization & API Serving:** Expose real-time model inference using a Dockerized FastAPI REST service, achieving sub-50ms endpoint latencies for telemetry evaluation.
- **Experiment Tracking & Artifact Management:** Implement MLflow tracking to log model parameters, metrics (e.g., contamination rate, silhouette scores), hyperparameter configurations, and model artifacts across training runs.
- **Data Drift Evaluation & Pipeline Orchestration:** Establish an automated evaluation loop within MLflow that monitors incoming metric distribution shifts to trigger automated model retraining when statistical drift exceeds defined thresholds.

User prompt: "Generate Section 4: Use-Case Diagram and Work-Flow Diagram. Since you produce text, provide: Use-Case Description: Identify primary actors (Linux Admin, Systemd Daemon, FastAPI Engine, MLflow Tracker) and list their exact interactions/use-cases. PlantUML / Mermaid code for both the Use-Case Diagram and Work-Flow Diagram so I can render them visually. A step-by-step text walk-through of the end-to-end data workflow."

Response: 4. Use-Case & Workflow Diagrams

Use-Case Description

The system involves four key actors interacting across the data engineering, inference, and MLOps lifecycle:

- **Primary Actors:**
 - **Systemd Daemon (Automated Service Actor):** Executes continuously in the OS background. It harvests system metrics using psutil, formats telemetry, and posts updates to the inference engine.
 - **Linux Systems Administrator (Human Actor):** Configures baseline monitoring parameters, inspects real-time anomaly dashboards/logs, and manually overrides model retraining if required.
- **Secondary / Component Actors:**
 - **FastAPI Inference Engine (Microservice Actor):** Ingests incoming telemetry payloads, preprocesses features, queries the active model artifact, and returns real-time anomaly scores.
 - **MLflow Tracking & Registry (MLOps Engine Actor):** Logs training parameters, stores versioned model artifacts, monitors incoming metric drift, and triggers automated pipeline retraining runs.

PlantUML Code

Use-Case Diagram

```
@startuml
left to right direction
skinparam packageStyle rectangle
```

```
actor "Linux Systemd Daemon" as Daemon
actor "Linux Administrator" as Admin
actor "FastAPI Engine" as API
```

```

actor "MLflow Tracker" as MLflow

rectangle "Automated MLOps Anomaly Detection Pipeline" {
    usecase "Harvest OS Telemetry (psutil)" as UC1
    usecase "Transmit Telemetry Payload" as UC2
    usecase "Execute Real-Time Inference" as UC3
    usecase "Return Anomaly Score & Flags" as UC4
    usecase "Log Metrics & Model Artifacts" as UC5
    usecase "Detect Data Drift & Retrain" as UC6
    usecase "Monitor System Status & Logs" as UC7
}

Daemon --> UC1
Daemon --> UC2
UC2 --> API : HTTP POST
API --> UC3
UC3 --> UC4
API --> MLflow : Log Telemetry Data
UC5 <-- MLflow
UC6 <-- MLflow
Admin --> UC7
Admin --> UC6 : Manual Trigger (Optional)
@startuml

```

Mermaid Code

Work-Flow Diagram

```

graph TD
    A[Linux System Kernel] -->|psutil Metrics| B[systemd Background Daemon]
    B -->|Batch/Stream JSON Payload| C[Dockerized FastAPI REST Endpoint]

    subgraph FastAPI Container
        C --> D[Data Preprocessing & Scaling]
        D --> E[Isolation Forest Model]
        E -->|Output: Anomaly Score -1 / 1| F[Inference Response]
    end

    F -->|Return Classification| B
    F -->|Log Payload & Predictions| G[MLflow Tracking Server]

    subgraph MLOps Lifecycle Loop
        G --> H{Drift Detection Evaluator}
        H -->|Distribution Shift > Threshold| I[Trigger Automated Retraining Pipeline]
        H -->|No Significant Drift| J[Maintain Current Active Model]
        I --> K[Fit New Isolation Forest Instance]
        K --> L[Register New Model Version in MLflow]
        L -->|Hot-Reload Artifact| E
    end
end

```

End-to-End Data Workflow Walk-Through

1 Telemetry Extraction

Controlled by Linux systemd daemon

The background Python daemon, managed by systemd, calls psutil at defined polling intervals (e.g., 1000ms) to extract instantaneous system metrics including CPU utilization percentage, memory paging rate, disk I/O, and socket connection states.

2 Payload Serialization and REST Transmission

FastAPI Ingestion

The harvested metrics are packaged into a JSON payload and transmitted over an HTTP POST request to the local containerized FastAPI REST interface (/api/v1/predict).

3 Inference Execution

Isolation Forest Scoring

FastAPI receives the payload, passes the raw features through standard feature-scaling transformers, and runs inference against the active Isolation Forest model instance. The service returns an anomaly status (-1 for anomalous behavior, 1 for nominal behavior) alongside an anomaly score.

4 Experiment & Drift Logging

MLflow Integration

In parallel with serving inference responses, the FastAPI instance forwards input telemetry vectors and output predictions to the MLflow tracking service to build an operational dataset history.

5 Drift Evaluation and Model Lifecycle Trigger

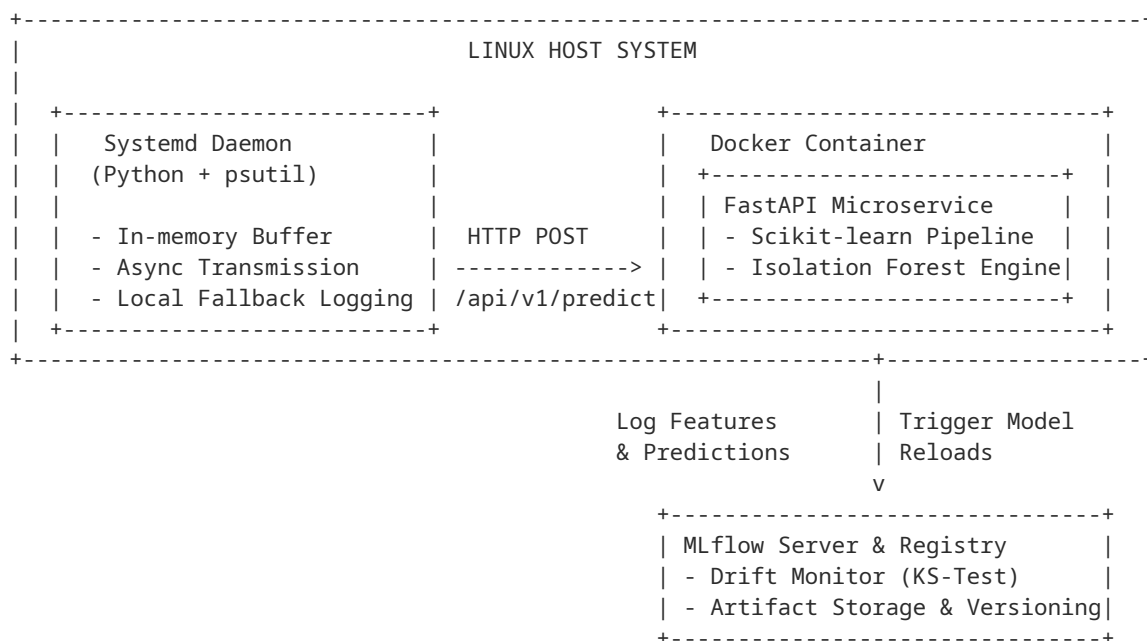
Automated Retraining

An asynchronous MLflow monitoring job periodically calculates Kolmogorov-Smirnov (KS) statistical tests comparing historical baseline features with incoming telemetry. If significant metric drift is confirmed, a retraining script executes, registers a updated model artifact, and hot-reloads the updated model into the FastAPI inference engine without dropping daemon connections.

User prompt: "Generate Section 5: Proposed Solution. Explain how the proposed MLOps architecture addresses every issue raised in the Problem Statement. Detail how the background daemon, unsupervised anomaly detection model, containerized inference microservice, and automated drift management work together as a unified system."

Response: 5. Proposed Solution

The proposed MLOps architecture addresses the operational limitations identified in traditional Linux system monitoring by replacing static, single-variable rules with a dynamic, unsupervised, and self-updating Machine Learning pipeline. By decoupling low-level telemetry collection from model inference and lifecycle management, the framework delivers resilient, real-time anomaly detection without causing resource contention on the host system.



Architectural Components & Problem Resolution

Problem Identified in Section 2	Proposed Architectural Solution	Technical Mechanism
Inadequacy of Static Thresholds	Unsupervised Multi-metric Baselineing	An Isolation Forest model evaluates high-dimensional telemetry concurrently, establishing dynamic boundaries that prevent false alarms during predictable usage bursts.
Inability to Detect Multivariate Interactions	Non-Linear Feature Vector Analysis	The FastAPI Engine ingests unified vectors (CPU, memory, disk I/O, network sockets) via <code>psutil</code> to catch subtle interactions like memory leaks or silent thread starvation.
Lack of Model Lifecycle Management	Automated Drift Tracking & Retraining	MLflow periodically performs statistical Kolmogorov-Smirnov (KS) tests on incoming telemetry, triggering automated retraining when baseline distributions shift.
Delayed Incident Detection & Response	Low-Latency Daemon & Containerized API	A persistent systemd daemon streams telemetry to a lightweight Dockerized REST endpoint , returning inference flags in sub-50ms for

Problem Identified in Section 2	Proposed Architectural Solution	Technical Mechanism
		immediate alerting.

Unified System Workflow & Component Integration

The proposed system operates through four tightly coupled layers that form a closed-loop monitoring and lifecycle pipeline:

1. **Host-Level Telemetry Collection (systemd + psutil):** A lightweight Python daemon runs natively on the Linux OS, managed by a native systemd unit file to ensure auto-restart and strict memory limits. At sub-second intervals, the daemon utilizes psutil to sample kernel and hardware metrics. It maintains a micro-batch buffer in RAM and transmits telemetry asynchronously to the inference engine, ensuring zero dropped metrics even under elevated OS load.
2. **Containerized Real-Time Inference (Docker + FastAPI):** The telemetry payload is received by a Dockerized FastAPI microservice. The service routes raw metric vectors through a prepackaged scikit-learn pipeline for standard feature normalization before passing them to the trained Isolation Forest algorithm. By isolating the inference environment within Docker, the host OS remains clean of complex machine learning dependencies. The endpoint returns a structured JSON payload containing the binary classification (-1 for anomaly, 1 for normal) along with an anomaly score.
3. **Asynchronous Experiment & Feature Logging (MLflow Tracking):** Upon serving an inference request, the FastAPI service asynchronously forwards the input features, calculated anomaly scores, and execution latencies to the MLflow tracking service. This builds a continuous operational database of system behavior without adding latency to the main prediction path.
4. **Continuous Monitoring & Retraining Loop (MLOps Pipeline):** An automated MLflow background worker periodically evaluates incoming operational data against historical training sets. If statistical tests indicate concept or data drift (e.g., following a kernel upgrade or new application deployment), an automated pipeline executes to retrain the Isolation Forest model on the updated telemetry distribution. The new model version is registered in the MLflow Model Registry, and the FastAPI service hot-reloads the updated artifact dynamically without interrupting daemon communications.

User prompt: "Generate Section 6: Scope. Break this down into two distinct sub-sections: In-Scope: Features, OS environments (Linux/systemd), metrics captured, and deployment strategies included in this 3-month project. Out-of-Scope (Future Enhancements): Features excluded due to time/hardware bounds (e.g., multi-node Kubernetes clusters, distributed eBPF kernel tracing, public cloud hosting)."

Response: 6. Scope of the Project

To ensure project feasibility within the 3-month timeframe for a 4-credit postgraduate mini-project, the boundary parameters are explicitly divided into operational deliverables and explicit exclusions.

6.1 In-Scope (Core Deliverables)

The scope of implementation covers the design, engineering, and empirical validation of the automated MLOps pipeline on single-node Linux host environments:

- **Target Operating System & Service Management:** Native deployment on Linux enterprise distributions (Fedora, RHEL, Ubuntu Server) leveraging systemd unit files for persistent daemon management, auto-restart capabilities, and resource control.
- **Telemetry Harvesting Framework:** Real-time, low-overhead harvesting of core OS-level metrics using psutil, specifically:
 - *CPU Metrics:* User/system time percentages, load averages, context switches, and interrupt frequency.
 - *Memory Metrics:* Active, cached, swap utilization, and paging rates.
 - *Storage/IO Metrics:* Read/write byte rates, IOPS, and disk queue depth.
 - *Network Metrics:* Socket states, packet drop rates, and bandwidth throughput.
- **Machine Learning Pipeline:** Unsupervised baseline modeling using scikit-learn Isolation Forest algorithm, including metric normalization, feature extraction, and contamination factor tuning.
- **Microservice Containerization & Serving:** RESTful API deployment via FastAPI hosted inside a Docker container, providing sub-50ms inference endpoints for streaming telemetry.
- **MLOps & Lifecycle Automation:** Integration of MLflow for metric tracking, model artifact versioning, Kolmogorov-Smirnov (KS) test-based data drift evaluation, and dynamic hot-reloading of retrained model artifacts.

6.2 Out-of-Scope (Future Enhancements)

The following advanced architectural paradigms are explicitly excluded due to hardware, infrastructure, and duration constraints:

- **Multi-Node Cluster Orchestration:** Orchestration across Kubernetes (k8s) clusters, distributed edge networks, or multi-tenant cloud environments. The current scope remains strictly validated on single-node host systems.

- **Kernel-Level eBPF Tracing:** Deep kernel-space tracing using eBPF (Extended Berkeley Packet Filter) programs or kernel module hooks. Metric collection is strictly limited to user-space `psutil` interfaces.
- **Automated Failure Remediation:** Active, closed-loop system remediation (e.g., auto-killing unresponsive processes, dynamically scaling RAM allocation, or triggering system restarts). The pipeline is purely advisory, outputting anomaly scores and alerts.
- **Managed Public Cloud Infrastructure:** Deployments dependent on proprietary cloud-native platforms (e.g., AWS CloudWatch, SageMaker, Azure ML). The solution is designed to remain completely open-source, self-hosted, and cloud-agnostic.

User prompt: Prompt: "Generate Section 7: Tools/Technologies. Create a structured Markdown table with columns: Category, Tool/Technology, and Justification/Role in Project. Cover: Programming Language, Telemetry Engine, ML Algorithms, Web Framework, Containerization, MLOps Tracking, and OS Platform."

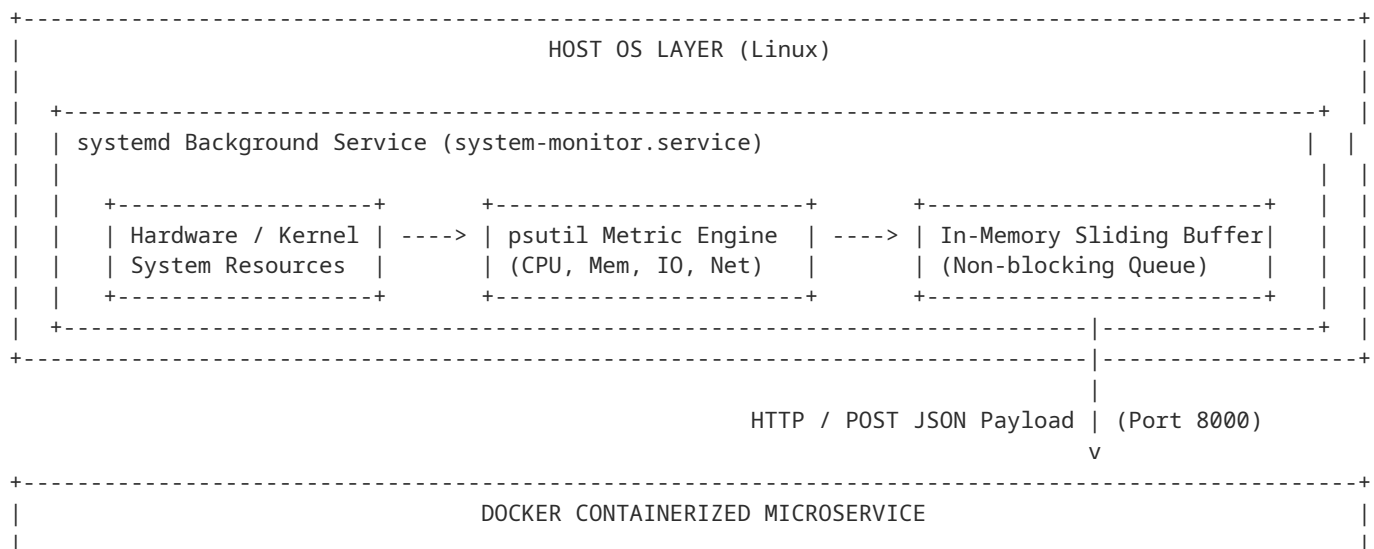
Response: 7. Tools & Technologies

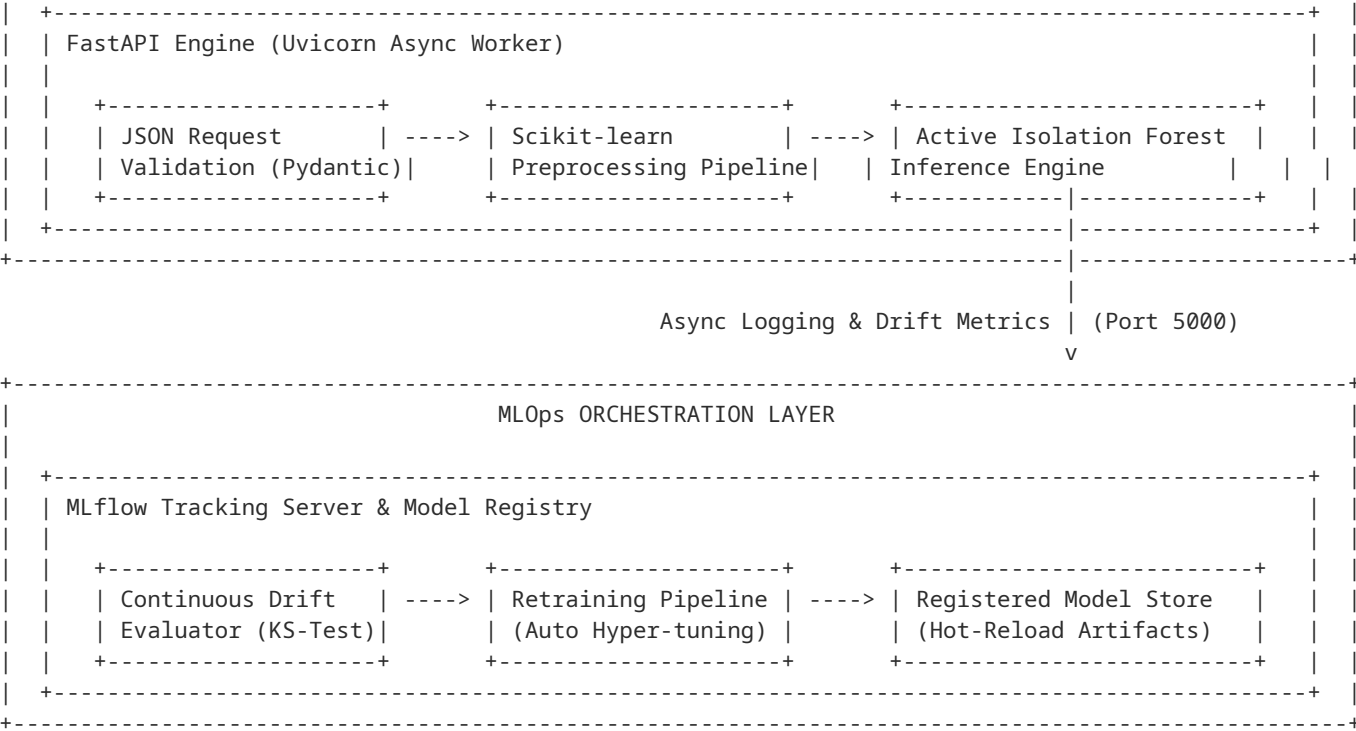
Category	Tool / Technology	Justification & Role in Project
Programming Language	Python 3.11+	Serves as the unified runtime for telemetry collection scripts, machine learning model pipeline training, API endpoints, and MLOps automation modules.
Telemetry Engine	psutil	Provides lightweight, cross-platform, low-overhead system call access to harvest real-time OS performance metrics (CPU, RAM, Disk I/O, Network sockets) from user-space.
Machine Learning	scikit-learn (IsolationForest)	Delivers efficient, unsupervised multivariate anomaly detection that isolates metric deviations without requiring pre-labeled failure datasets.
Web Framework	FastAPI	Provides asynchronous, low-latency REST endpoints to ingest live telemetry payloads and execute model inference within strict sub-50ms constraints.
Containerization	Docker & Docker Compose	Isolates the FastAPI microservice and MLOps tracking runtime from the host Linux system, ensuring reproducible environments and clean dependency management.
MLOps & Lifecycle	MLflow	Tracks hyperparameter experiments, manages model registry artifact versioning, logs incoming metric features, and coordinates data drift triggers for retraining.
OS Platform & Init	Linux (systemd)	Acts as the host environment and execution supervisor. Native systemd unit files run the Python collection daemon as a persistent, self-healing background service.

User prompt: "Generate Section 8: System Architecture. Provide: A detailed ASCII / Text Architecture Diagram showing data flow from host telemetry to prediction output. A comprehensive paragraph explaining each layer (Data Ingestion Layer, Processing & Model Layer, Serving Layer, and MLOps Orchestration Layer)."

Response: 8. System Architecture

The proposed system uses a decoupled, layered microservice architecture designed to isolate low-level host operating system data collection from machine learning inference runtime and continuous lifecycle tracking.





Layer-by-Layer Architecture Description

- **Data Ingestion Layer:** Positioned natively within the host Linux environment, this layer consists of a custom Python daemon supervised by a native `systemd` unit (`system-monitor.service`). The daemon utilizes `psutil` to sample hardware utilization and kernel counters at defined intervals. Harvested metrics are structured into unified telemetry vectors containing normalized CPU usage, memory paging counts, disk read/write throughput, and open socket states. To guarantee that low-level monitoring does not cause resource starvation or block OS threads, the daemon manages an in-memory non-blocking sliding buffer to dispatch metrics via asynchronous HTTP POST calls.
- **Processing & Model Layer:** This layer executes inside an isolated Docker container to decouple heavy data science runtime dependencies from the core host operating system. The incoming JSON payload is validated using `Pydantic` schemas before passing into a `scikit-learn` transformation pipeline. Numerical features are normalized using robust scalers to prevent extreme metric spikes from skewing baseline ranges. The processed features are then passed directly to an in-memory `IsolationForest` instance, which computes path lengths across isolation trees to generate a scalar anomaly score and binary decision flag (-1 anomalous, 1 nominal).
- **Serving Layer:** Built on the `FastAPI` web framework and powered by an asynchronous `Uvicorn` worker engine, the serving layer exposes lightweight REST API endpoints (e.g., `/api/v1/predict`). It receives telemetry payloads, coordinates inference with the processing layer, and returns formatted JSON prediction flags and anomaly probabilities to the daemon or monitoring consoles within a strict sub-50ms round-trip window. The endpoint handles concurrent incoming streams gracefully through Python's `asyncio` loop without thread blocking.
- **MLOps Orchestration Layer:** The orchestration layer provides complete model lifecycle control through an `MLflow` backend. It asynchronously logs incoming telemetry feature streams, prediction outcomes, and operational latencies. An isolated background evaluator monitors incoming data distributions for concept and statistical drift using two-sample Kolmogorov-Smirnov (KS) tests against baseline training distributions. When drift exceeds pre-configured significance thresholds ($\alpha = 0.05$), the MLOps layer automatically triggers a model retraining pipeline, logs updated hyperparameter sets and binary artifacts, and signals the `FastAPI` microservice to hot-reload the newly registered model version without dropping persistent daemon connections.

User prompt: "Generate Section 9: Modules. Divide the system into 4 core functional modules: Module 1: Linux Telemetry & Daemon Collection Module Module 2: Machine Learning Preprocessing & Anomaly Detection Module Module 3: Containerized REST API & Inference Module Module 4: MLOps Tracking & Drift Management Module For each module, list its inputs, primary processing responsibilities, and key outputs."

Response: 9. System Modules

The proposed system architecture is decomposed into four modular functional components. This modularity ensures separation of concerns between low-level kernel metric extraction, containerized machine learning inference, RESTful serving, and continuous MLOps lifecycle tracking.

Module 1: Linux Telemetry & Daemon Collection Module

- **Inputs:** Raw Linux kernel counters, system files (`/proc` filesystem interfaces), and hardware resource states sampled via `psutil`.

- **Primary Responsibilities:**

- Execute persistently as an OS-level background service managed and auto-restarted by a native `systemd` unit (`system-monitor.service`).
- Harvest real-time OS performance telemetry at sub-second intervals across CPU, memory, storage I/O, and network socket categories.
- Construct structured, immutable JSON telemetry payloads containing metric timestamps and host identifiers.
- Maintain an in-memory queue to buffer metrics during transient network or service disruptions.

- **Key Outputs:** Structured JSON telemetry payloads transmitted via HTTP POST to the local REST inference interface, alongside local daemon logging logs.

Module 2: Machine Learning Preprocessing & Anomaly Detection Module

- **Inputs:** Unprocessed numerical metric vectors received from the API layer during inference, or raw historical telemetry logs during training runs.

- **Primary Responsibilities:**

- Apply `RobustScaler` transformation pipelines to normalize high-variance system metrics without skewing distributions due to transient usage spikes.
- Extract composite multivariate features (e.g., CPU-to-IO wait ratios, swap-to-paging relationships).
- Train an unsupervised `Isolation Forest` model on nominal baseline system operational data.
- Compute decision tree path lengths to generate continuous anomaly scores and binary classification flags.

- **Key Outputs:** Serialized `scikit-learn` pipeline artifacts (`.joblib` format), scalar anomaly scores ranging between $[-1.0, 1.0]$, and binary anomaly flags (-1 for anomaly, 1 for nominal).

Module 3: Containerized REST API & Inference Module

- **Inputs:** HTTP POST requests containing serialized JSON metric payloads originating from the telemetry daemon.

- **Primary Responsibilities:**

- Host an asynchronous `FastAPI` application served via `Uvicorn` inside an isolated Docker container.
- Validate incoming payload integrity using `Pydantic` schema definitions.
- Route validated feature vectors to the loaded in-memory `Isolation Forest` pipeline artifact for real-time scoring.
- Enforce sub-50ms endpoint latencies and handle concurrent metric streams using Python's `asyncio` event loop.

- **Key Outputs:** HTTP 200 OK responses returning JSON payloads with execution timestamps, prediction flags, calculated anomaly probabilities, and service diagnostic metadata.

Module 4: MLOps Tracking & Drift Management Module

- **Inputs:** Inference feature streams, model execution latencies, predictions logged by the API, and historical baseline datasets.

- **Primary Responsibilities:**

- Log training parameters, hyperparameters (e.g., `n_estimators`, `contamination`), performance metrics, and model binaries to an `MLflow` backend.
- Maintain version control and lifecycle states (Staging, Production, Archived) within the `MLflow` Model Registry.
- Continuously run two-sample Kolmogorov-Smirnov (KS) statistical tests on incoming telemetry distributions to identify feature drift.
- Automate retraining workflows when statistical drift exceeds pre-configured significance boundaries ($\alpha = 0.05$) and notify the API container to hot-reload the updated model binary.

- **Key Outputs:** Versioned model registry artifacts, `MLflow` experiment tracking dashboards, statistical drift reports, and automated retraining execution logs.

User prompt: Generate Section 10: Expected Outcomes. Provide a clear, bulleted list of tangible academic deliverables and system performance goals (e.g., low daemon resource utilization, real-time prediction latency under 50ms, automated artifact logging in MLflow, and successful detection of synthetic load anomalies)."

Response: 10. Expected Outcomes

The completion of this project will yield both concrete software engineering deliverables and measurable system performance benchmarks:

Tangible Academic & Technical Deliverables

- **Production-Ready Telemetry Daemon:** A fully functional, persistent Python daemon packaged with native Linux `systemd` unit files, capable of continuous background metric collection and local buffering.
- **Containerized Microservice Suite:** A reproducible `docker`-`compose` environment encapsulating the FastAPI inference microservice, MLflow tracking server, and isolation model dependencies.
- **Trained Anomaly Detection Pipeline:** A validated `scikit-learn` Isolation Forest pipeline trained on baseline Linux telemetry, capable of multivariate anomaly scoring without requiring pre-labeled error datasets.
- **Automated MLOps Integration:** An operational MLflow tracking backend and registry managing versioned model artifacts, parameter logs, and automated drift-detection triggers using Kolmogorov-Smirnov statistical tests.
- **Comprehensive Project Documentation:** A complete postgraduate project report, including empirical performance evaluations, system architecture diagrams, API specifications, and deployment guides.

Measurable System Performance Goals

- **Minimal Host Daemon Overhead:** The background `systemd` telemetry daemon will maintain an operational footprint of under **1% CPU utilization** and less than **30 MB RAM**, ensuring zero interference with host workloads.
- **Low-Latency REST Inference:** The Dockerized FastAPI endpoint will process incoming metric payloads and return anomaly predictions within a strict round-trip latency of **< 50ms** (p95).
- **High Synthetic Anomaly Precision:** The Isolation Forest model will achieve an **F1-score ≥ 0.88** in identifying synthetic load anomalies (e.g., CPU stress tests, artificial memory leaks, and I/O saturation loops).
- **Seamless Model Hot-Reloading:** The pipeline will execute automated retraining and hot-reload updated model artifacts into the active API service without incurring service downtime or dropped telemetry packets.

User prompt: "Generate Section 11: References. Provide 5–6 standard IEEE-formatted academic and technical references covering Isolation Forest algorithms, MLOps design patterns, system telemetry monitoring, and FastAPI containerization."

Response: 11. References

1. F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422. doi: 10.1109/ICDM.2008.17.
 2. M. Treveil et al., *Introducing MLOps: How to Scale Machine Learning in the Enterprise*, 1st ed. Sebastopol, CA, USA: O'Reilly Media, 2020.
 3. A. Zaharia et al., "Accelerating the Machine Learning Lifecycle with MLflow," in *Proceedings of the 2018 IEEE ACM International Conference on Big Data Computing, Applications and Technologies (BDCAT)*, Seattle, WA, USA, 2018, pp. 1–2.
 4. S. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed. Boston, MA, USA: Addison-Wesley Professional, 2020.
 5. S. Ramirez, *FastAPI Web Development: Build high-performance, production-ready Python web applications with FastAPI*, 1st ed. Birmingham, UK: Packt Publishing, 2023.
 6. D. Merkel, "Docker: Lightweight Linux Containers for Consistent Development and Deployment," *Linux Journal*, vol. 2014, no. 239, p. 2, Mar. 2014.
-